(http://mng.bz/wvK2), "Auto-Instrumenting Node.js Apps With OpenTelemetry" (http://mng.bz/qrpJ), "Trace Context Propagation With OpenTelemetry" (http://mng.bz/7DAe), and "OpenTelemetry in Java: Tutorial & Agent Example" (https://www.containiq.com/post/opentelemetry-in-java) are all excellent resources. Next up, we have a look at the OpenTelemetry collector—the "universal signal type" agent or "universal telemetry" agent.

4.3.2 Collector

OpenTelemetry's agent is called the *collector*. It comprises pipelines that assemble collector components, such as receivers, processors, and exporters, as depicted in figure 4.6. Think of a pipeline as a combination of components that define how and where you get the signals from (receivers), process them at the edge (transform, batch, etc., via processors and connectors), and into which backend to ingest the signals (exporters).

You can define one or more pipelines in a collector. Each pipeline is dedicated to a signal type (e.g., metrics). You can also have multiple pipelines for the same signal type, such as different environments or to enforce different policies. Each of the pipelines acts independently, collecting signals from the sources and ingesting them into backends.

The native way to talk to the collector is via the OpenTelemetry Protocol (OTLP; https://tinyurl.com/mngbz), a general-purpose telemetry data-delivery protocol. The OTLP specification describes the encoding, transport, and delivery mechanisms of signals between the downstream sources, intermediate nodes (like collectors), and backend destinations.

OTLP itself was designed by the OpenTelemetry project and is an incremental improvement over the OpenCensus protocol, focusing on high reliability of delivery and clear visibility when the data cannot be delivered. In addition, OTLP is L7-load-balancer friendly, supporting the correct mapping of imbalanced incoming traffic to balanced outgoing traffic. Further, OTLP allows backpressure signaling from the destinations all the way back to sources, enabling you to build out reliable multihop delivery via intermediate nodes (e.g., OpenTelemetry collectors), each with potentially different processing capacities.

From a protocol perspective, OTLP uses a request–response style: the downstream clients send requests, and the upstream server replies with corresponding responses. OTLP is rather flexible—you can run it over gRPC, HTTP, and (going forward) WebSockets.

While in the future, we may live in a world where everyone and everything natively supports OpenTelemetry—that is, speaks OTLP—this is not yet the case at the time of writing, in 2023. The OpenTelemetry community found a clever way to help jump-start things: by introducing a vast array of ingress and egress components (receivers and exporters) that can read existing formats and protocols from sources as well as write existing formats and protocols to the destinations.